cornea_simple_mech_model

Hyperelastic and poroelastic finite-element simulations built on FEniCSx (dolfinx 0.11), for two kinds of mesh:

- **2D axisymmetric**: a meridian section (r, z) of the cornea under intraocular pressure (IOP),
- **3D**: any gmsh volume mesh (tetrahedra or hexahedra), with a unit-cube test case.

The `Simulation_Framework` package is a thin layer over dolfinx. It handles:

- mesh reading (physical tags and an optional per-cell local reference system),
- material laws and weak forms (hyperelastic, and poroelastic with incompressible constituents),
- boundary conditions and the Newton solver,
- outputs (XDMF fields, CSV scalars, metadata) and plots.

A simulation is a short script: a mesh, a JSON parameter file, a list of boundary conditions and the quantities you want to record. The same script structure works in 2D axisymmetric and in 3D, and for both physics; only the framework class changes. The frameworks are verified against analytical solutions (section 10).

Author: B. Sauty

---

0. Pdf compile

In the terminal, to obtain a pdf version of this readme, run :

```
pandoc README.md -s -o README.pdf -V geometry:margin=2cm
```

1. Installation (Docker)

FEniCSx is run inside a Docker container built from the provided `Dockerfile`. The image and the container are both named `fenicsx_cornea`.

Prerequisites

- Docker (Docker Desktop on Windows / macOS)

Build the image

From the root of the repository:

```
docker build -t fenicsx_cornea .
```

This starts from `dolfinx/dolfinx:v0.11.0` (dolfinx, PETSc, MPI, gmsh) and adds a LaTeX installation (matplotlib text rendering), OpenGL / Xvfb libraries (pyvista off-screen rendering) and the Python packages pandas, scipy, sympy, matplotlib, h5py, imageio, pyvista, pytest, jupyter and spyder-kernels.

Create and start the container

The repository is mounted as `/workspace`, so files are shared both ways: results written by the container appear in your local folder.

Linux / macOS:

```
docker run -it --name fenicsx_cornea -p 8888:8888 \
    -v "$(pwd)":/workspace fenicsx_cornea
```

Windows (PowerShell):

```
docker run -it --name fenicsx_cornea -p 8888:8888 `
    -v "${PWD}:/workspace" fenicsx_cornea
```

Come back to the same container later with `docker start -ai fenicsx_cornea`. Check the installation:

```
python3 -c "import dolfinx; print(dolfinx.__version__)"
```

Python path

The scripts import `Simulation_Framework` and `Meshing_tools` as top-level packages, so the repository root must be on `PYTHONPATH`. The Dockerfile sets `ENV PYTHONPATH=/workspace`; outside Docker, use `export PYTHONPATH=$PWD` from the repository root.

Jupyter (optional)

```
jupyter notebook --ip=0.0.0.0 --port=8888 --no-browser --allow-root
```

Then open the `http://127.0.0.1:8888/?token=...` link printed in the terminal.

Post-processing without Docker

`PlotManager` and the results reader do **not** need dolfinx or MPI. On a machine without Docker, a plain Python environment is enough:

```
pip install numpy matplotlib h5py pillow
```

Building a PDF of this file

```
pandoc README.md -s -o README.pdf -V geometry:margin=2cm
```

The `geometry` option matters: with pandoc's default page layout the text block is too narrow for the tables and their columns overlap. The same option applies to `Testing_tools/Check_against_analytical/Terzaghi_equation.md`.

---

2. Repository structure

```
cornea_simple_mech_model/
|-- Dockerfile, pytest.ini, LICENSE, README.md
|-- Simulation_Framework/                 # the library
|   |-- __init__.py                       # lazy exports
|   |-- hyperelastic.py                   # Hyperelastic_framework + axisymmetric / 3D classes
|   |-- poroelastic.py                    # Poroelastic_framework + axisymmetric / 3D classes
|   |-- geometry.py                       # Axisymmetric, Cartesian3D, push_local_frame
|   |-- materials.py                      # HyperelasticMaterial, PorousMaterial, PoroelasticMaterial
|   |-- solver.py                         # PETSc/SNES options, make_problem, solve_and_report
|   |-- quantities.py                     # build_quantities: standard output quantities
|   |-- mesh_io.py                        # read_mesh / write_mesh: tags + local reference system
|   `-- postprocessing_tools/
|       |-- log_utils.py                  # log: printing in MPI runs
|       |-- outputs_manager.py            # OutputManager (needs dolfinx)
|       |-- output_utils.py               # point_probe, average (need dolfinx)
|       |-- results_reader.py             # reads XDMF/HDF5/CSV/JSON (no dolfinx)
|       `-- plots_manager.py              # PlotManager, compare_scalars (no dolfinx)
|-- Meshing_tools/                        # mesh generators (gmsh)
|   |-- mesh_generation.py, coons.py, structured_cylinder.py   # shared tools
|   |-- make_axisym_cornea.py, make_3D_cornea.py               # cornea meshes
|   |-- make_axisym_rectangle.py, make_cube.py                 # test meshes
|   `-- Meshes/                           # default output folder of the generators
|-- Usecases/                             # one folder per case: script, mesh, parameters, plots
|   |-- cornea_axi/                       # hyperelastic, axisymmetric
|   |-- cornea_3D/                        # hyperelastic, 3D
|   |-- cornea_poro_axi/                  # poroelastic, axisymmetric
|   `-- cube/                             # hyperelastic, 3D unit cube
|-- Testing_tools/
|   |-- test_mesh_generation.py, test_meshio.py   # pytest
|   |-- check_LRS_parallel.py                     # run with mpirun (section 5)
```

```
|   `-- Check_against_analytical/                # verification scripts (section 10)
`-- results/                                  # created by the runs, mirrors the folders above
```

---

3. Architecture and public API

Physics and geometry are separate

Each physics is written **once**, in one class, for both geometries:

File	Class written once	Physics
`hyperelastic.py`	`Hyperelastic_framework`	displacement $\mathbf{u}$ (P2), strain energy $\psi(\mathbf{F})$
`poroelastic.py`	`Poroelastic_framework`	displacement $\mathbf{u}$ (P2) and pore pressure p (P1), mixed

What differs between the axisymmetric and the 3D case is provided by a small **geometry object** (`geometry.py`) that the class reads as `self.geometry`:

Operator	`Axisymmetric`	`Cartesian3D`
`grad_vector(u, x)`	3×3 gradient in (r, z, θ), hoop term u_r/r	`ufl.grad(u)`
`grad_scalar(p, x)`	$(\partial_r p, \partial_z p, 0)$	`ufl.grad(p)`
`normal_product(A, N)`	(r, z) block of $\mathbf{A} \cdot \mathbf{N}$	$\mathbf{A} \cdot \mathbf{N}$
`weight(x)` (multiplies `dx`, `ds`)	r	1
`integral_factor`	2π (full revolved body)	1
`axes`, `axisymmetric`	`(r, z, t)`, `True`	`(x, y, z)`, `False`

The four public classes are two-line subclasses that only choose the geometry:

```python
class Poroelastic_3D_framework(Poroelastic_framework):
    geometry = Cartesian3D()
```

Support modules shared by both physics: `materials.py` (parameters and laws), `solver.py` (Newton options and the step report), `quantities.py` (the standard output quantities), `mesh_io.py`.

Public API

Everything is importable from the package root:

```python
from Simulation_Framework import (Hyperelastic_axisymmetric_framework, Hyperelastic_3D_framework,
                                  Poroelastic_axisymmetric_framework, Poroelastic_3D_framework,
                                  read_mesh, write_mesh, MeshData, top_cell_centroids,
                                  HyperelasticMaterial, PoroelasticMaterial,
                                  OutputManager, point_probe, average, log,
                                  PlotManager, compare_scalars)
```

Class	Problem
`Hyperelastic_axisymmetric_framework`	Hyperelastic, 2D (r, z) axisymmetric
`Hyperelastic_3D_framework`	Hyperelastic, 3D
`Poroelastic_axisymmetric_framework`	Poroelastic, 2D (r, z) axisymmetric
`Poroelastic_3D_framework`	Poroelastic, 3D

Name	Module	Role
`read_mesh`, `MeshData`	`mesh_io`	Reads a `.msh`: dolfinx mesh, facet and cell tags, local reference system
`write_mesh`	`mesh_io`	Writes a mesh with its tags and local frame
`top_cell_centroids`	`mesh_io`	Cell centroids, used by the mesh generators
`HyperelasticMaterial`	`materials`	Solid parameters (validated dataclass) and the laws
`PoroelasticMaterial`	`materials`	Solid and porous parameters
`OutputManager`	`outputs_manager`	Writes fields (XDMF) and scalars (CSV) at each step, plus a metadata file
`point_probe`, `average`	`output_utils`	Helpers to build case-specific scalar outputs
`log`	`log_utils`	Prints on rank 0, in order with the PETSc output
`PlotManager`	`plots_manager`	Reads the result files and plots them
`compare_scalars`	`plots_manager`	Overlays several runs

Imports are **lazy**: a module is loaded only when one of its names is used, so `from Simulation_Framework import PlotManager` works without dolfinx.

How the pieces fit

```
 .msh + params.json --> Framework --(quantities, surface/volume_integral)--> case script
                                                                                  |
                                                                                  v
                                                                           OutputManager
                                                                                  |
                                  <basename>.xdmf/.h5   <basename>_scalars.csv   <basename>_meta.json
                                                                                  |
                                                                                  v
                                                                  results_reader --> PlotManager
```

The framework knows the physics, `OutputManager` knows nothing about it, and `PlotManager` only reads files. The sidecar `<basename>_meta.json` is what lets the plots know the coordinate labels and whether the run is axisymmetric.

---

4. Usage

All commands are run **from the repository root** (the scripts use paths such as `Usecases/cornea_axi/cornea.msh` and write to `results/...`). Simulations run inside the container.

Use cases

Each folder of `Usecases/` holds its `main_*.py` script, its parameter file(s), its mesh and its plot script.

Folder	Framework	Problem
`cornea_axi`	hyperelastic, axisymmetric	IOP ramp 0 to 20 mmHg on the posterior face, limbus clamped, axis $u_r = 0$
`cornea_3D`	hyperelastic, 3D	Same load on a 3D hexahedral cornea (coarse and fine meshes)
`cornea_poro_axi`	poroelastic, axisymmetric	Constant IOP of 18 mmHg, 100 time steps up to 1000 s, drained limbus, flux on the posterior face (Giammarini 2026)
`cube`	hyperelastic, 3D	Unit cube clamped on three faces, follower load on `z1` ramped to 500 mmHg

```
python3 Usecases/cornea_axi/main_cornea_axi.py
mpirun -n 4 python3 Usecases/cornea_axi/main_cornea_axi.py      # in parallel
python3 Usecases/cornea_axi/plot_cornea.py                      # figures, no dolfinx needed
python3 Usecases/cornea_3D/main_cornea_3D.py
python3 Usecases/cornea_poro_axi/main_cornea_poro_axi.py
python3 Usecases/cube/main_cubic_test.py
python3 Usecases/cube/plot_cubic.py
```

Each run writes, in `results/Usecases/<folder>/`: `<name>.xdmf` / `.h5` (fields), `<name>_scalars.csv` (scalars) and `<name>_meta.json` (metadata). In the hyperelastic runs the first column of the outputs, `t`, is the **load** (pressure); in the poroelastic runs it is the **time** (s). Step 0 is the reference state.

Tests and verification

```
python3 -m pytest                                              # mesh generation and mesh I/O
python3 Testing_tools/Check_against_analytical/check_poro_axi_MR_terzaghi.py   # one check
mpirun -n 2 python3 Testing_tools/check_LRS_parallel.py                        # LRS in parallel
```

`pytest` runs `Testing_tools/test_*.py`. The scripts of `Check_against_analytical` are not pytest tests: each runs a few simulations, prints a table of errors against an analytical solution and saves a plot (section 10).

Units are consistent everywhere: lengths in mm, stresses in MPa (1 mmHg = 1.33322e-4 MPa).

Axisymmetric vs 3D at a glance

	axisymmetric classes	3D classes
Mesh	2D, $r \geq 0$	3D
Displacement space	P2 vector, 2 components	P2 vector, 3 components
Gradient	∇u plus hoop term u_r/r	∇u
Integrals	weighted by r; `volume_integral` / `surface_integral` include 2π (full revolved body)	plain integrals
Output axes	(r, z, t), `t` = hoop	(x, y, z)
Dirichlet axis	$0 = r$, $1 = z$	$0 = x$, $1 = y$, $2 = z$
Tensor outputs	3×3 in (r, z, θ)	3×3 in (x, y, z)
Extra quantity	`hoop_stretch` $= 1 + u_r/r$, attribute `mech.r`	none
Field plots	yes	no (ParaView)

---

5. Mesh

`read_mesh(mesh_file, comm)` opens a gmsh file once and returns a `MeshData` with `domain`, `facet_tag`, `cell_tags` and the optional local reference system `e_1`, `e_2`, `e_3`. The geometric dimension (2 or 3) is taken from the file. The frameworks call it for you. `write_mesh` writes a mesh with its tags and local frame (used by the generators).

Physical tags

Cornea, 2D (`Usecases/cornea_axi/cornea.msh`):

Tag	Name	Default boundary condition in `main_cornea_axi.py`
1	`anterior`	free (zero traction)
2	`posterior`	follower pressure (IOP)
3	`limbus`	clamped in r and z
4	`central_line`	symmetry axis: $u_r = 0$
10	`cornea`	the domain

Cornea, 3D (`cornea_3D_hex_*.msh`): 1 `anterior`, 2 `posterior`, 3 `limbus`.

Cube (`Usecases/cube/cube_*.msh`, side 1, 5 cells per side for the hexahedral mesh):

Tag	Face	Tag	Face
1	`x0`	4	`y1`
2	`x1`	5	`z0`
3	`y0`	6	`z1`
10	the volume `cube`		

Rectangle section (written by `make_axisym_rectangle.py`, used by the verification checks of the axisymmetric framework): 1 `top`, 2 `bottom`, 3 `lateral`, 4 `central_line`, 10 `rectangle`. For a hollow cylinder, `central_line` is the inner surface $r = r_i$.

Local reference system (LRS)

A mesh may carry three per-cell orthonormal vector fields as gmsh `$ElementData` views, stored by the reader as DG0 functions with 3 components:

View name (either spelling)	2D (axisymmetric)	3D
`e_1` / `e1`	read from the file	read from the file
`e_2` / `e2`	read from the file	read from the file
`e_3` / `e3`	$(0, 0, 1)$, the hoop direction	read from the file if present, else $e_1 \times e_2$

The LRS is optional. A mesh without `e_1` and `e_2` is valid (`MeshData.has_lrs` is `False`), and isotropic laws do not need it. Fibre laws (HGO) raise an error without it. Views may cover all elements or only the cells; boundary elements are skipped.

> Parallel behaviour of the LRS (ghost cells) has not been verified; it is correct on a single rank. `Testing_tools/check_LRS_parallel.py` writes `lrs_<n>.npy` for `n` ranks, to compare runs with different numbers of ranks.

Mesh generators

`Meshing_tools` builds structured meshes from a unit block (square or cube) deformed by a map ϕ, with the local frame pushed forward by ϕ (`mesh_generation.py`). The map of the cornea is a Coons patch (`coons.py`); the 3D cornea uses a structured cylinder (`structured_cylinder.py`).

Script	Writes (in `Meshing_tools/Meshes/`)
`make_axisym_cornea.py`	axisymmetric cornea section
`make_3D_cornea.py`	3D hexahedral cornea (`cornea_3D_hex.msh`)
`make_axisym_rectangle.py`	rectangle section `[r_i, r_e] x [0, h]`
`make_cube.py`	cube (`cube_hexa.msh`, or tetrahedra with `simplex=True`)

```
python3 Meshing_tools/make_cube.py
```

Each use case keeps its own copy of the mesh it runs on (in its folder), so copy the mesh you want from `Meshing_tools/Meshes/` to the use case after generating it.

---

6. Material parameters

Parameters are read from a JSON file by the material class of the framework (`HyperelasticMaterial.from_json`, `PoroelasticMaterial.from_json`).

Hyperelastic laws

```
{
  "sedf_type": "Mooney-Rivlin",
  "C_10": 0.5,
  "C_01": -0.175,
  "D": 0.4
}
```

`sedf_type`	Parameters
`Neo-Hookean`	`C_10`, `D`
`Mooney-Rivlin`	`C_10`, `C_01`, `D`
`HGO`	`C_10`, `C_01`, `D`, `k1`, `k2`, `kappa`, `a4`, `a6`

The strain energy densities are

$$\psi_{\mathrm{NH}} = C_{10}(\bar{I}_1 - 3) + \frac{(J-1)^2}{D},$$

$$\psi_{\mathrm{MR}} = C_{10}(\bar{I}_1 - 3) + C_{01}(\bar{I}_2 - 3) + \frac{(J-1)^2}{D},$$

$$\psi_{\mathrm{HGO}} = \psi_{\mathrm{MR}} + \frac{k_1}{2k_2} \sum_{i \in \{4,6\}} \mathcal{H}(E_i) \left(e^{k_2 E_i^2} - 1 \right),$$

with

- $\bar{\mathbf{C}} = J^{-2/3}\mathbf{F}^T\mathbf{F}$, $\bar{I}_1 = \operatorname{tr}\bar{\mathbf{C}}$, $\bar{I}_2 = \frac{1}{2}\left[(\operatorname{tr}\bar{\mathbf{C}})^2 - \operatorname{tr}(\bar{\mathbf{C}}^2)\right]$,
- $E_i = \kappa(\bar{I}_1 - 3) + (1 - 3\kappa)(\bar{I}_i - 1)$ and $\bar{I}_i = \mathbf{a}_i \cdot \bar{\mathbf{C}}\,\mathbf{a}_i$ (generalised structure tensor with dispersion κ, on the isochoric part of $\mathbf{C}$),
- $\mathcal{H}(E_i)$ the Heaviside function: fibres only carry load in extension ($E_i > 0$).

D controls compressibility: the initial bulk modulus is $2/D$ and $D \to 0$ is the incompressible limit. The initial shear modulus is $\mu = 2(C_{10} + C_{01})$.

Fibres. `a4` and `a6` are 3-component vectors given **in the mesh's local reference system**, e.g. `[1, 0, 0]` is along e_1. For an HGO law the framework builds the fibre fields itself when it is created: the directions are pushed to physical space cell by cell and normalised, and a mesh without a local reference system is rejected at that point. Parameters not used by the chosen law are ignored.

Validation (`HyperelasticMaterial.__post_init__`): `sedf_type` must be one of the three laws, the required parameters must be present, $D > 0$, $k_1 \geq 0$, $k_2 > 0$, $\kappa \geq 0$, and `a4`, `a6` must be non-zero 3-vectors.

Poroelastic material

The solid parameters are the same as above (Neo-Hookean or Mooney-Rivlin), plus the porous ones:

```
{
  "sedf_type": "Mooney-Rivlin", "C_10": 0.5, "C_01": -0.175, "D": 0.4,
  "porosity_type": "Isotropic", "phi_s": 0.2, "k": 1.5e-3
}
```

Parameter	Meaning
`porosity_type`	`"Isotropic"` (the only one)
`phi_s`	solid fraction, in $(0, 1]$

Parameter	Meaning
`k`	permeability of the Darcy law $\mathbf{w} = -k\nabla p$ (mm^2 MPa^{-1} s^{-1} in the units of the repository)

Formulation (incompressible constituents, Giammarini 2026). Unknowns are $\mathbf{u}$ (P2) and p (P1), solved together:

- skeleton and fluid: $\mathbf{P} = \phi_s\, \partial\psi_{\text{iso}}/\partial\mathbf{F} - Jp\,\mathbf{F}^{-T}$, with ψ_{iso} the **isochoric part** of the strain energy only,
- mass balance: $\dot{J}\, q + (\mathbf{K}\nabla p)\cdot\nabla q$ integrated over the body, with the pulled-back permeability $\mathbf{K} = J\,\mathbf{F}^{-1}(k\mathbf{I})\mathbf{F}^{-T}$ and $\dot{J} \simeq (J - J_n)/\Delta t$ (backward Euler; J_n comes from the previous step).

Consequences: the parameter `D` is **not used** by the poroelastic frameworks (the volume change is carried by the fluid), and the drained constrained modulus is $M = \frac{4}{3}\,\phi_s\,\mu$. The HGO law is not available in the poroelastic frameworks (`psi_iso` is not defined for it).

---

7. Writing a simulation

Both physics have the same interface; only the framework class and the boundary condition names change. The order is: framework, `build_weak_form`, `build_BCs`, `build_solver`, **then** the outputs. `surface_integral` needs the measure `mech.ds` created by `build_BCs`.

Hyperelastic

```python
import numpy as np
from dolfinx import fem
from petsc4py.PETSc import ScalarType

from Simulation_Framework import Hyperelastic_3D_framework, OutputManager

# 1. framework: reads the mesh once (and builds the fibre fields for HGO), then the weak form
mech = Hyperelastic_3D_framework("Usecases/cube/cube_hexa.msh",
                                 "Usecases/cube/mech_params_cube.json")
mech.build_weak_form()

# 2. boundary conditions, then the solver
p = fem.Constant(mech.domain, ScalarType(0.0))
mech.build_BCs([
    ["Dirichlet", 1, ("clamped", 0)],          # face x0: u_x = 0
    ["Dirichlet", 3, ("clamped", 1)],          # face y0: u_y = 0
    ["Dirichlet", 5, ("clamped", 2)],          # face z0: u_z = 0
    ["Neumann_follower", 6, p],                # face z1: follower pressure p
])
mech.build_solver()

# 3. outputs: the case decides what is recorded
q = mech.quantities                            # standard UFL quantities
fields  = {"displacement": q["displacement"], "cauchy_stress": q["cauchy_stress"]}
scalars = {"volume": mech.volume_integral(q["J"])}

# 4. load stepping
with OutputManager("results/my_run", mech.domain, fields, scalars,
                   meshtags={"facet_tag": mech.facet_tag},
                   meta=mech.output_meta) as out:
    out.write(0.0)                             # reference state
    for n, load in enumerate(np.linspace(0, 1e-2, 11)[1:], start=1):
        p.value = load
```

```
        mech.solve_one_step(n, load)
        out.write(load)                          # the "time" of the outputs is the load
history = out.get_history()                      # scalars as numpy arrays
```

Poroelastic

The weak form needs the time step as a `fem.Constant`, updated before each step:

```
from Simulation_Framework import Poroelastic_3D_framework

mech = Poroelastic_3D_framework(mesh_file, "Usecases/cornea_poro_axi/mech_params_Giammarini2026.json")
dt = fem.Constant(mech.domain, ScalarType(0.0))
mech.build_weak_form(dt)

p0 = fem.Constant(mech.domain, ScalarType(1e-3))
mech.build_BCs([
    ["Dirichlet_disp", 1, ("clamped", 0)],       # as in the hyperelastic case
    ["Dirichlet_disp", 5, ("clamped", 2)],
    ["Neumann_follower_disp", 6, p0],            # load on the top face
    ["Dirichlet_pressure", 6, "drained"],        # drained top, other faces impervious
])
mech.build_solver()

# ... outputs as above, then, with t_list the times (t_list[0] = 0):
#     for n, t in enumerate(t_list[1:], start=1):
#         dt.value = t - t_list[n - 1]
#         mech.solve_one_step(n, t)
#         out.write(t)
```

`Usecases/cornea_poro_axi/main_cornea_poro_axi.py` is a complete example, with the flux condition `Neumann_follower_pressure`.

Framework attributes you will use: `domain`, `facet_tag`, `cell_tags`, `u` (and `p` for poroelastic), `mech_params`, `quantities`, `output_meta`, `volume_integral(expr)`, `surface_integral(expr, tag)`.

Boundary conditions

`build_BCs(boundary_conditions, slip_penalty=1e3)` takes a list of `[type, tag, values]`. The facet tags come from the mesh, `mech.facet_tag`. Each `Dirichlet` entry constrains **one** component: clamp a face fully with one entry per axis.

Hyperelastic

Type	values	Meaning
`Dirichlet`	`("clamped", axis)` or `(Constant, axis)`	Imposed displacement component
`Neumann_follower`	`Constant` p	Follower pressure normal to the deformed surface, $p > 0$ pushes into the body
`Neumann`	traction vector	Dead load (reference area)
`Slip`	`None` or a constant vector $\mathbf{m}$	Sliding: $\mathbf{u} \cdot \mathbf{N} = 0$ (or $\mathbf{u} \cdot \mathbf{m} = 0$), by a penalty of stiffness `slip_penalty` $\times \mu/h$
`Robin`	`(k, u_ref)`	Elastic support of stiffness k

Poroelastic (displacement conditions end with `_disp`, pressure ones with `_pressure`)

Type	`values`	Meaning
`Dirichlet_disp`	`("clamped", axis)` or `(Constant, axis)`	Imposed displacement component
`Neumann_disp`	traction vector	Dead load
`Neumann_follower_disp`	`Constant` p	Follower pressure, $p > 0$ pushes into the body
`Slip_disp`	`None` or a constant vector	Sliding, by penalty
`Robin_disp`	`(k, u_ref)`	Elastic support
`Dirichlet_pressure`	scalar, `Constant`, callable, or `"drained"`	Imposed pore pressure (0 for `"drained"`)
`Neumann_pressure`	flux q_n	Outward fluid flux per reference area
`Neumann_follower_pressure`	flux q_n	Outward fluid flux per current (deformed) area

A face with no pressure condition is impervious. Vector values are lists or UFL vectors with 2 components (axisymmetric) or 3 (3D), and may contain `Constant`s.

Solver

`build_solver(**petsc_overrides)` sets up a PETSc SNES Newton solver (`newtonls`, no line search, LU with MUMPS, absolute tolerance 10^{-14}, relative 10^{-10}) with `quadrature_degree` 4 and P2 displacement. The options live in `solver.py` (`DEFAULT_PETSC_OPTIONS`); pass overrides to change them for one simulation, e.g. `mech.build_solver(snes_rtol=1e-8)`. The absolute tolerance is deliberately small: with stresses in MPa the residuals can be of order 10^{-8}, and a larger value makes SNES accept the initial guess without iterating.

`solve_one_step(n, load)` solves one step, prints the iteration count and residual, and raises `RuntimeError` if SNES does not converge. The previous step is the initial guess of the next one, so ramp the load (or the time) in steps small enough for Newton to converge.

---

8. Outputs

Standard quantities

`mech.quantities` is a dict of symbolic UFL expressions, built by `build_quantities` (nothing is compiled until `OutputManager` gets them); `print(mech.quantities.keys())` lists them.

Name	Meaning
`displacement`	$\mathbf{u}$ (P2 function)
`PK1_stress`	first Piola-Kirchhoff stress $\mathbf{P}$ (for poroelastic, including the pressure term)
`cauchy_stress`	$\boldsymbol{\sigma} = J^{-1}\mathbf{P}\mathbf{F}^T$
`green_lagrange`	$\mathbf{E} = \frac{1}{2}(\mathbf{F}^T\mathbf{F} - \mathbf{I})$
`J`	volume ratio
`von_mises`	$\sqrt{\frac{3}{2}\,\mathrm{dev}\boldsymbol{\sigma} : \mathrm{dev}\boldsymbol{\sigma}}$
`hydrostatic_pressure`	$-\mathrm{tr}\,\boldsymbol{\sigma}/3$
`hoop_stretch`	$1 + u_r/r$ (axisymmetric only)
`fluid_pressure`	pore pressure p (poroelastic only)

OutputManager

```
OutputManager(basename, domain, fields=None, scalars=None, meshtags=None, meta=None)
```

It is generic: it does not know the physics, the dimension or the tags. The **caller defines** what is recorded.

- **fields**: `{name: expression}` or `{name: (expression, (family, degree))}`. The default space is `("Lagrange", 1)` for a `fem.Function` (so `displacement` is written as a nodal P1 field) and `("DG", 0)` for any other UFL expression (cellwise constant). Vector and tensor shapes are taken from the expression; spaces are shared between fields.
- **scalars**: `{name: ufl.Form}` (integral, compiled once, summed over ranks; use `mech.volume_integral` / `mech.surface_integral`) or `{name: callable}` (zero-argument function returning a float, called on all ranks). A ratio of two integrals is not a form: use `average` (below) or a callable.
- **meshtags**: `{name: MeshTags}` written to the XDMF file, usually `{"facet_tag": mech.facet_tag}`.
- **meta**: pass `mech.output_meta`. Written to the sidecar JSON together with `gdim` and `tdim`.

`write(t)` interpolates and writes all fields (one XDMF time step) and appends one CSV row, flushed immediately so rows survive a later failing step. It must be called on all ranks. Use it as a context manager or call `close()`.

Files written: `<basename>.xdmf` (+ `.h5`), `<basename>_scalars.csv` (first column `t`), `<basename>_meta.json`. Missing parent folders are created.

Helpers for case-specific scalars

Built on dolfinx; both are collective (call on all ranks).

```
from Simulation_Framework import point_probe, average

# value of a function at a fixed reference point (2D or 3D): all components
u_apex = point_probe(mech.domain, mech.u, (0.0, z_apex))
scalars["apex_uz"] = lambda: u_apex()[1]

# mean of a quantity over a surface (or a volume)
one = fem.Constant(mech.domain, ScalarType(1.0))
scalars["uz_top"] = average(mech.surface_integral(q["displacement"][2], 6),
                            mech.surface_integral(one, 6), mech.domain.comm)
```

Forces, volumes and energies from the axisymmetric classes are for the full revolved body (integrated over 2π). The limbus reaction in `main_cornea_axi.py` is an example of a surface integral of $\mathbf{P} \cdot \mathbf{N}$.

Examples of case definitions: `cornea_outputs` in `Usecases/cornea_axi/main_cornea_axi.py` (apex displacements, central thickness, limbus reaction, volume), `cube_outputs` in `Usecases/cube/main_cubic_test.py`, and the `*_outputs` functions of the verification checks (mean strains and stresses, pressure profiles).

---

9. Plotting

`PlotManager` reads the result files only (numpy, h5py, matplotlib). It adapts to the run from `<basename>_meta.json`: coordinate labels and component names are `r`, `z`, `t` (hoop) for an axisymmetric run and `x`, `y`, `z` otherwise.

```
import matplotlib.pyplot as plt
from Simulation_Framework import PlotManager, compare_scalars

pm = PlotManager("results/Usecases/cornea_axi/cornea_IOP_Pandolfi",
                 labels={"apex_uz_anterior": r"anterior apex $u_z$"})
pm.info()                                          # what is in the files

# scalar histories: vs load t, or one scalar vs another (any dimension)
pm.plot_scalars(["apex_uz_anterior", "apex_uz_posterior"])
pm.plot_scalars("limbus_reaction_z", x="apex_uz_anterior")

# maps on the (r, z) section (2D only)
pm.plot_field("von_mises", deformed=True, mirror=True)
pm.plot_field("cauchy_stress", component="tt", symmetric=True)   # hoop stress
```

```python
# profiles along the symmetry axis: last step, then all steps
pm.plot_profile("green_lagrange", component="zz")
pm.plot_profile("green_lagrange", component="zz", t="all")

pm.animate_field("von_mises", "vm.gif", deformed=True, mirror=True)

# parameter study
runs = [PlotManager(f"results/run_{k}", label=f"run {k}") for k in range(3)]
compare_scalars(runs, "apex_uz_anterior")
plt.show()
```

Method	Purpose	Dimension
`info()`	Prints dimension, axes, scalars, fields and their time range	any
`plot_scalars(y, x="t")`	One or several scalars against t or another scalar	any
`plot_field(name, ...)`	Colour map at one load step	2D
`sample(name, points, ...)`	Field values at reference points	2D
`plot_profile(name, ...)`	Field along a line (default: the symmetry axis)	2D
`animate_field(name, ...)`	`.gif` (Pillow) or `.mp4` (ffmpeg) over all steps	2D
`compare_scalars(managers, y)`	Overlay runs	any
`quick_look(basename)`	One panel per scalar (call `plt.show()` after)	any

Common arguments

- `t`: the load step (default: the last one; the nearest stored step is used, with a warning if it differs).
- `component`: vectors take an axis label (`'r'`, `'z'`, `'x'`...) or `'magnitude'`; tensors take two labels (`'rr'`, `'zz'`, `'tt'`, `'rz'`, `'xy'`...).
- `deformed=True, scale=...`: draw on the deformed configuration (needs the `displacement` field).
- `mirror=True`: also draw the $r < 0$ half (axisymmetric runs only).
- `symmetric=True`: colour range centred on 0 with a diverging colormap.
- `ax=...`: draw into your own axis (every method returns its axis); `save="file.png"`: create, save and close a stand-alone figure. Do not combine the two. Nothing calls `plt.show()` for you.
- Profiles in a non-axisymmetric run need explicit `start` and `end` points.

Label names for case-specific scalars go in `PlotManager(labels={...})`; the standard quantities are labelled by the library (`DEFAULT_LABELS`).

3D results

Field maps, profiles and animations are 2D only: `PlotManager` raises `NotImplementedError` on a 3D run. Scalar histories work in 3D, and the fields are meant to be opened in ParaView from `<basename>.xdmf` (the facet tags are written there too). `results_reader` already reads 3D meshes (tetrahedra, hexahedra), so 3D plotting can be added without touching the file format.

Reading results directly

`results_reader` (numpy and h5py only) exposes `read_scalars`, `read_meta`, `XDMFResults` (mesh, cells, fields, time series, `values(name, t)`) and `extract_component`, for custom post-processing.

---

10. Verification against analytical solutions

`Testing_tools/Check_against_analytical/` holds one script per problem with a known solution. Each one creates its meshes (or reads the cube), writes the parameter files and runs a few simulations in a fixed folder `results/Testing_tools/Check_against_analytical/<Check_name>/`, then prints a table of relative errors and saves a plot (analytical solution as lines, simulation as markers). Run them from the repository root, in the container.

The script names are `check_<name>.py`; the name says the physics (`hyper`, `poro`), the geometry (`axi`, `3D`) and the law.

`<name>`	Problem	Analytical reference
`hyper_axi_MR_axial`	Filled cylinder, axial load ramp, free lateral surface	Homogeneous uniaxial state of a compressible Mooney-Rivlin law (root of $\sigma_{rr} = 0$)
`hyper_3D_MR_axial`	Cube, same loading, in compression and in tension	Same
`hyper_axi_MR_pressure`	Hollow cylinder under internal pressure, sliding ends	Finite-strain Lamé problem, equilibrium ODE solved by shooting; compares p, mean J, axial force, radial profile
`hyper_axi_HGO_axial`	HGO fibres radial+hoop and radial+axial, tension and compression	Homogeneous state with fibres; checks the tension-only switch (`hgo_homogeneous.py`). Radial+axial in compression has no homogeneous solution and is skipped
`hyper_3D_HGO_axial`	HGO fibres x+y and x+z, tension and compression	Same
`poro_axi_MR_terzaghi`	Poroelastic: confined column consolidating under a constant load, drained top	Terzaghi: bottom pressure, degree of consolidation and pressure profiles against $T = c_v t/H^2$
`poro_3D_MR_terzaghi`	Poroelastic: same on the cube	Same

- The analytical solution is evaluated at the stretch (or inner radius) returned by the simulation, so the comparison does not depend on the sign or amplitude of the load. For the poroelastic checks all cases fall on the same curve in T.
- The HGO checks and `hgo_homogeneous.py` assume the fibre energy of section 6.
- The equations of the Terzaghi checks are in `Terzaghi_equation.md` (`pandoc Terzaghi_equation.md -s -o Terzaghi_equation.pdf`).
- The mesh of the cube checks is `Usecases/cube/cube_hexa.msh` (5 cells per side). The poroelastic 3D check chooses its first time step from the number of elements along z (`N_ELEM_Z`): the diffusion length $\sqrt{T}H$ must span at least two elements.

---

11. Extending the library

- **New output**: no registry to edit. Add an entry to the `fields` / `scalars` dicts of your case, or a new key to `build_quantities` (`quantities.py`) to share it between cases; if it depends on a feature of the framework, test that feature there (`output_meta["axisymmetric"]`, `hasattr(framework, "p")`). Add a label in `DEFAULT_LABELS` (`plots_manager.py`) or in `PlotManager(labels=...)`.
- **New material law**: add the name to `SedfType` and `REQUIRED_SOLID` in `materials.py`, and a branch in `HyperelasticMaterial.strain_energy_density_function` returning $\psi(\mathbf{F})$. All four frameworks use it unchanged, since $\mathbf{P} = \partial\psi/\partial\mathbf{F}$ (the poroelastic ones use its isochoric part).
- **New boundary condition**: add one branch in `build_BCs` of the physics class (`hyperelastic.py` or `poroelastic.py`). Write it with the geometry operators (`geo.normal_product`, the weight `w`) and it works in axisymmetric and 3D.
- **New geometry**: a class with the members of the table in section 3, and a two-line subclass of each framework.
- **New check**: copy the closest script of `Check_against_analytical`; the structure is outputs, `run_simulation`, analytical solution, cases, comparison and plots.

---

12. Known limitations

- **`D` and the poroelastic frameworks**: `D` is read from the JSON but not used (no volumetric skeleton energy), and nothing warns about it.

- **HGO in poroelastic**: not available (`psi_iso` raises `NotImplementedError`).
- **Time integration**: backward Euler, first order; the poroelastic use case uses uniform steps. Newton runs without line search.
- **Permeability**: constant isotropic `k`, not strain-dependent.
- **`tdim` vs `gdim`**: `read_mesh` takes the dimension from the file; a 2D mesh used with a 3D framework (or the reverse) is not rejected explicitly.
- **LRS in parallel**: not verified (see section 5).
- **3D plots**: field plots are 2D only (use ParaView for 3D fields).
- `plots_manager.py` has no command-line entry point, use `quick_look(basename)`.